

```
twinkle_twinkle.notes.add(pitch=65, start_time=4.0, end_time=4.5, velocity=80)
twinkle_twinkle.notes.add(pitch=65, start_time=4.5, end_time=5.0, velocity=80)
twinkle_twinkle.notes.add(pitch=64, start_time=5.0, end_time=5.5, velocity=80)
twinkle_twinkle.notes.add(pitch=64, start_time=5.5, end_time=6.0, velocity=80)
twinkle_twinkle.notes.add(pitch=62, start_time=6.0, end_time=6.5, velocity=80)
twinkle_twinkle.notes.add(pitch=62, start_time=6.5, end_time=7.0, velocity=80)
twinkle_twinkle.notes.add(pitch=60, start_time=7.0, end_time=8.0, velocity=80)

twinkle_twinkle.total_time = 8
twinkle_twinkle.tempos.add(qpm=60);

# This is a colab utility method that visualizes a NoteSequence.
mm.plot_sequence(twinkle_twinkle)

# This is a colab utility method that plays a NoteSequence.
mm.play_sequence(twinkle_twinkle, synth=mm.fluidsynth)
```

Machine learning with Magenta

The *magenta.music* library enables the developer to do the following:

- Create music using utilities in the Magenta library, by using abstractions.
- Generate music using ML models.
This library incorporates various ML models that are built using TensorFlow. Three of the prominent models are listed below.
- **MelodyRNN:** Given a note sequence, it will continue in the original style.
- **MusicVAE:** This is used to build novel NoteSequences. It can also be used to interpolate between sequences.
- **Onsets and frames:** This transcribes piano audio.

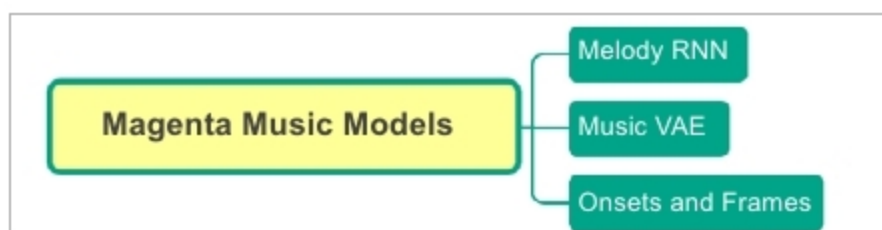


Figure 5: Magenta.Music models

Melody RNN

Melody RNN is primarily an LSTM based language model. It handles musical notes. The objective is to continue the given sequence in the original style. There are two steps to using it:

- Initialising the model
- Continuing a sequence

The code snippet for model initialising follows:

```
print('Downloading model bundle. This will take less than a minute...')
mm.notebook_utils.download_bundle('basic_rnn.mag', '/content/')

# Import dependencies.
from magenta.models.melody_rnn import melody_rnn_sequence_generator
from magenta.models.shared import sequence_generator_bundle
from magenta.music.protobuf import generator_pb2
from magenta.music.protobuf import music_pb2

# Initialize the model.
print("Initializing Melody RNN...")
bundle = sequence_generator_bundle.read_bundle_file('/content/basic_rnn.mag')
generator_map = melody_rnn_sequence_generator.get_generator_map()
melody_rnn = generator_map['basic_rnn'](checkpoint=None, bundle=bundle)
melody_rnn.initialize()
```

The sequence continuation can be carried out with two parameters – the number of steps and temperature. You can tweak these two parameters and understand the change in the resultant output:

```
# Model options. Change these to get different generated sequences!
input_sequence = twinkle_twinkle
num_steps = 128 # change this for shorter or longer sequences
temperature = 1.0 # the higher the temperature the more random the sequence.

# Set the start time to begin on the next step after the last note ends.
last_end_time = (max(n.end_time for n in input_sequence.notes)
                 if input_sequence.notes else 0)
qpm = input_sequence.tempos[0].qpm
seconds_per_step = 60.0 / qpm / melody_rnn.steps_per_quarter
total_seconds = num_steps * seconds_per_step

generator_options = generator_pb2.GeneratorOptions()
generator_options.args['temperature'].float_value = temperature
generate_section = generator_options.generate_sections.add(
    start_time=last_end_time + seconds_per_step,
    end_time=total_seconds)

# Ask the model to continue the sequence.
sequence = melody_rnn.generate(input_sequence, generator_options)

mm.plot_sequence(sequence)
mm.play_sequence(sequence, synth=mm.fluidsynth)
```